Széchenyi István Katolikus Technikum és Gimnázium

Szoftverfejlesztő és -tesztelő projektfeladat

„Gymbox”

Készítették:

Bohn Eliot Konstantin,

Pataki Márk,

Mészáros Gyula

Ózd, 2026

Tartalomjegyzék

Bevezetés ... 3
Rendszerkövetelmények ... 4
Webalkalmazás indítása ... 4
Főoldal leírása: ... 6
Regisztrációs oldal leírása: ... 9
Bejelentkezés oldal leírása: ... 10
Admin oldal leírás: ... 11
Bérlés oldal leírása: ... 12
Edzésterv készítés oldal leírása: ... 13
Kosár oldal leírása: ... 14
Saját bérléseim oldal leírása: ... 15
Fejlesztői dokumentáció: ... 17
Adatbázis ... 18
- Konténerek tábla: ... 18
- Felhasználó tábla: ... 18
- Gépek tábla: ... 19
- Gépcsomag tábla: ... 19
- Edzéstervek tábla: ... 20
- Csomagok tábla: ... 20
- Bérlés tábla: ... 20
- Kapcsolatok: ... 21

Kapcsolat ábrák: ... 21
EK modell ... 22
Backend: ... 23
- Tesztelések: ... 26

Frontend: ... 34
Tesztelés: ... 43
Továbbfejlesztési lehetőségek: ... 45
Irodalom jegyzék: ... 46

Bevezetés

A GymBox egy olyan webalkalmazás, amely fantasztikus megoldást kínál a fitnesz és az edzés területén. Az alkalmazás célja, hogy egyszerűbbé és rugalmasabbá tegye az edzőeszközök használatát azok számára, akik szeretnének rendszeresen mozogni anélkül, hogy hagyományos edzőteremi tagságot kellene vállalniuk, vagy fix helyhez lennének kötve. A GymBox segítségével különböző méretű és felszereltségű konténereket lehet bérelni, melyeket magas minőségű, korszerű edzőgépekkel töltöttünk fel, így akár otthon, akár a munkahely közelében, vagy bármilyen más, számukra kényelmes helyszínen végezhetik el a testmozgást megrendelőink.

Az alkalmazás célja nemcsak a sportolás könnyebbé tétele, hanem egy olyan közösség kialakítása is, ahol az emberek motiválják egymást, és személyre szabottabb módon érhetik el fitneszcéljaikat. A GymBox, felhasználói számára lehetővé teszi, hogy online gyorsan és egyszerűen válasszanak a különböző konténerek közül, amelyek eltérő edzésigényeket és felszereltségi szinteket kínálnak. A bérlés rugalmassága révén akár rövid időre is igénybe vehető egy-egy konténer, így a felhasználók saját időbeosztásuk szerint, stresszmentesen szervezhetik meg az edzéseiket.

Ez az újítás lehetőséget ad arra, hogy bárki, kortól és edzettségi szinttől függetlenül, saját tempójában és környezetében mozogjon, miközben egy professzionális és sokoldalú edzőfelszereléssel dolgozhat. A GymBox azoknak szól, akik értékelik a szabadságot és a személyre szabott megoldásokat, miközben nem akarnak lemondani a minőségi edzésről.

Mind hármunkhoz gyerekkora óta kötődik valamilyen sport. Ezért nem is volt kérdés, hogy valamiféle sport vagy fitnesz témában vágunk bele a projektbe. Több közös, hosszas ötletelés után az edző termi edzés került a középpontba. Mindannyiunk jártas a témában ugyanis a mindennapjaink részét képezi, de mi egy kicsit feldobtuk az „átlagos” helyszínt.

Link a projekthez: https://github.com/BohnEliot/Gymbox.git

Rendszerkövetelmények

Hardveres rendszerkövetelmények:

	Ajánlott	Minimális
Processzor (CPU)	Intel(R)_Core(TM)_i5-14600K_CPU_@_5.30GHz	Intel Core i3-6100T Dual-Core 3.2GHz
Operációs rendszer (OP)	Windows 11	Windows 10
RAM	16 GB Ram	6 GB RAM
Tárhely	20 GB	6 GB
Internet	Széles sávú	Széles sávú
GPU	NVIDIA GeForce GTX 1050	Intel(R) UHD Graphics 620

Szerveroldali követelmények:

- Operációs rendszer: Windows 10-11/Linux
- Backend: Node.js/PHP alapú szerver
- Hálózati kapcsolat: Minimum 100 Mbps internetkapcsolat

Kliensoldali követelmények:

- Megbízható böngésző

Webalkalmazás indítása

Az indításhoz szükséges szoftverek:

- Xampp 8.1.12 vagy újabb
- NodeJS v22.13.0 vagy újabb
- Composer2.8.4 vagy újabb
- Visual Studio Code v1.96.0 vagy újabb
- Bármilyen böngésző, amely naprakész és megbízható

Első lépés a backend beüzemelése. Ehhez szükséges telepíteni a Xampp-ot és a Composert. Megnyitjuk a xampp-control panelt és elindítjuk a Mysql és Apache modulokat, ez létrehozza a kapcsolatot a lokális adatbázissal, ahol az adatokat tároljuk. Második lépésként majd

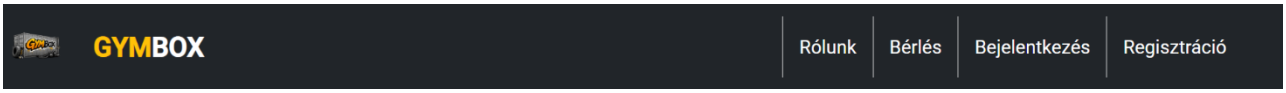
létrehozunk egy mappát pl.: Gymbox néven. A Gymbox mappában, ahol telepítve van az Angular CLI és a Laravel a Composerrel együtt, el kell helyezni a teljes backend és frontend mappákat, amelyek tartalmazzák az alkalmazás adatbázisát és felhasználói felületét. Harmadik lépésben elindítjuk a Visual Studio Code-ot és megnyitjuk benne a backend mappát, amely a php alapú Laravel keretrendszerben készült. Itt megnyitjuk a terminált (Ctrl + ö) és a következő parancsot írjuk be: php artisan migrate - -seed. Ezzel feltöltjük a xampp-ba az adatbázist. Aztán elindítjuk a szervert, szintén a terminálban a php artisan serve paranncsal. Ezt követően a Visual Studio Code-ban megnyitjuk a frontend mappát (GymboxAngular), amely Angular keretrendszerben készült. Itt a terminálban átváltunk parancssorra, a powershell helyett. Ha még nem telepítettük az Angulart, akkor beírjuk a terminálba, hogy: npm install -g @angular/cli. Ezzel feltelepítjük az adott mappába a keretrendszert. Majd beírjuk azt a parancsot, hogy ng serve -o. Ezzel megnyitottuk és elindítottuk a webalkalmazást.

Webalkalmazás használata

Mikor elindítottuk a webalkalmazást a főoldal tárul elénk. Itt több információ is megtalálható a Gymboxról. Egyből a bérlés gombbal találkozhatunk, ami átvezet a bérlés oldalra. Itt négy különböző méretű konténer közül lehet választani. Valamint 6 különböző konténer típusunk van. Csak kardió gépekkel felszerelt, csak erősítő gépekkel felszerelt, 25/75%-os felosztás vagy kardió vagy erősítő gépekkel felosztva, 50/50%-os felosztás. Mérettől és típustól függően egy előre összeállított csomagot lehet bérelni a konténerrel együtt, egy kiválasztott időtartamra (3 hónap, 1 év, 2 év stb.). De ehhez a bérléshez rendelkezni kell egy Gymbox fiókkal, amelyet a bejelentkezés/regisztráció oldalon tehetünk meg. Ha bejelentkeztünk vagy regisztráltunk egy eltárolt fiókunk lesz az adatbázisban. Innentől lehetséges a rendelés, amelyet, ha leadunk a rendeléseim oldalon meg lehet tekinteni részletes adatokkal ellátva, valamint a rendelés státuszát is nyomon követhetjük. Emellett lehet személyi edzőként is regisztrálni. A felület annyival bővül, hogy edzéstervet tud írni az edző és ezt a bérlő kiválaszthatja a konténerhez. Valamint van egy admin felület is. Itt az admin látja az edzésterveket és az összes rendelést és azok státuszát. Ha bármely fiókból ki akarunk lépni a „kijelentkezés” szövegre kattintva, amely a navigációs sávban megtalálható, ki tudunk jelentkezni.

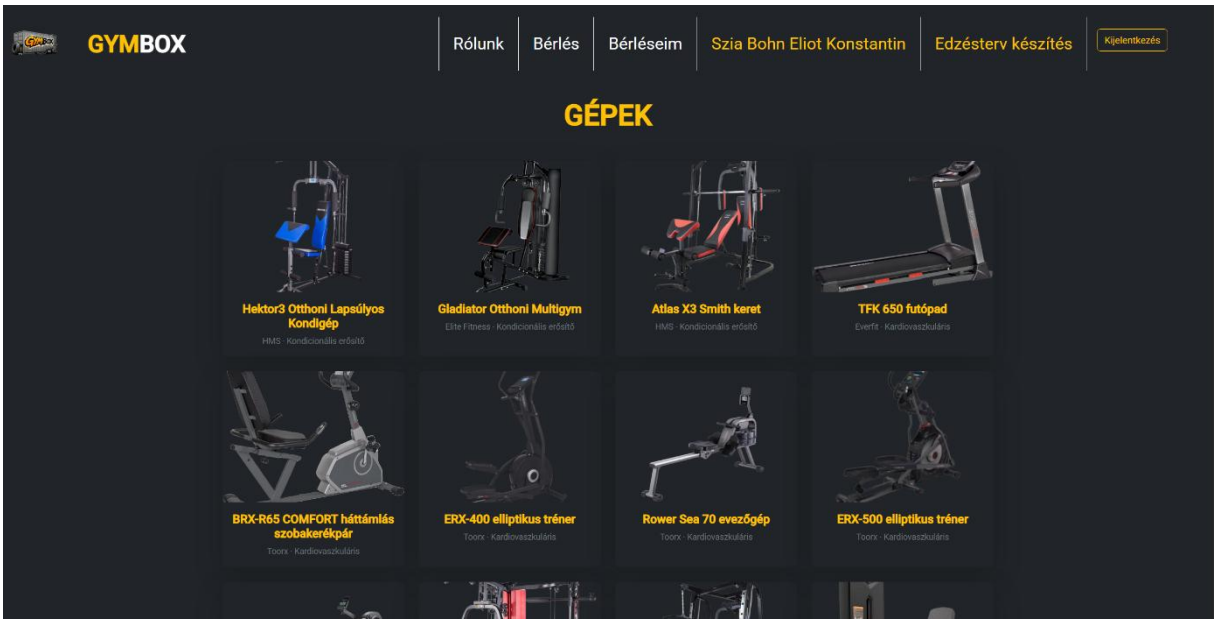
Főoldal leírása:

A főoldalon található egy navigációs sáv, ami egy külön Angular komponens és több oldalunkon is felhasználtuk az egység megőrzése érdekében, amely, ha nem vagyunk bejelentkezve az alábbi adatokat mutatja:

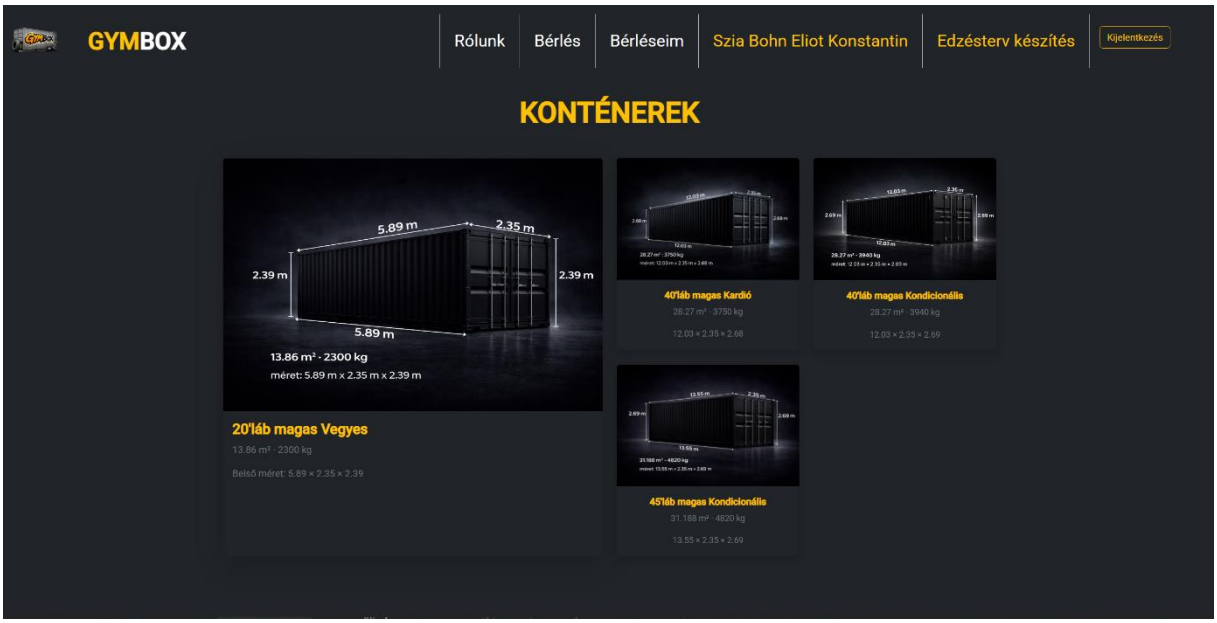


1.Gymbox navigációs sáv

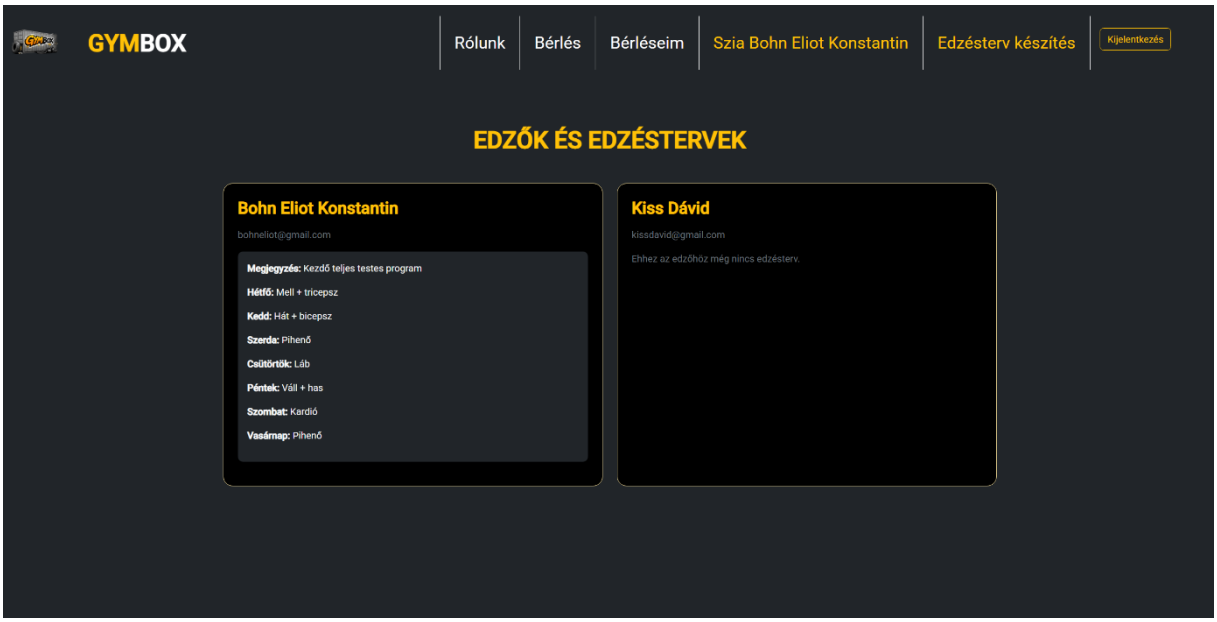
Megtalálható egy bérlés gomb a borítókép közepére igazítva, amely átvezet minket a bérlés oldalra, ahol megkezdhetjük a bérlést, ha rendelkezünk fiókkal. Három kártya található a gomb alatt, amelyekre kattintva az oldal különböző részeire irányít minket. Ez alatt szinként három kártyát láthatunk: gépeink, konténereink és edzőink. Ezekre kattintva külön oldalon megjelenítve láthatjuk ezeket.



2. Gépeink oldal



3. Konténereink oldal



4. Edzőink oldal

Majd egy sávban a Gymbox szállítási gyorsaságáról és megnyerő kapcsolat fenntartásáról bekezdések

5. Főoldal gymbox szállítás

Ezt követően ismét egy borítókép következik, amelyen a rendelj felirat szerepel. Alkalmazásunk fő célja, a konténerek összeállítása és kiszállítása, ezért az oldalon többször megjelenik ez.

6. Fő oldal rendelés oldalra irányítás

A következő része az oldalnak, az edzői regisztrációt ismerteti. Röviden leírást kaphatunk arról, hogy mit is jelent ez, ha edzőként regisztrálunk az oldalra. A gombra kattintva át is navigál a bejelentkezés / regisztráció oldalunkra.

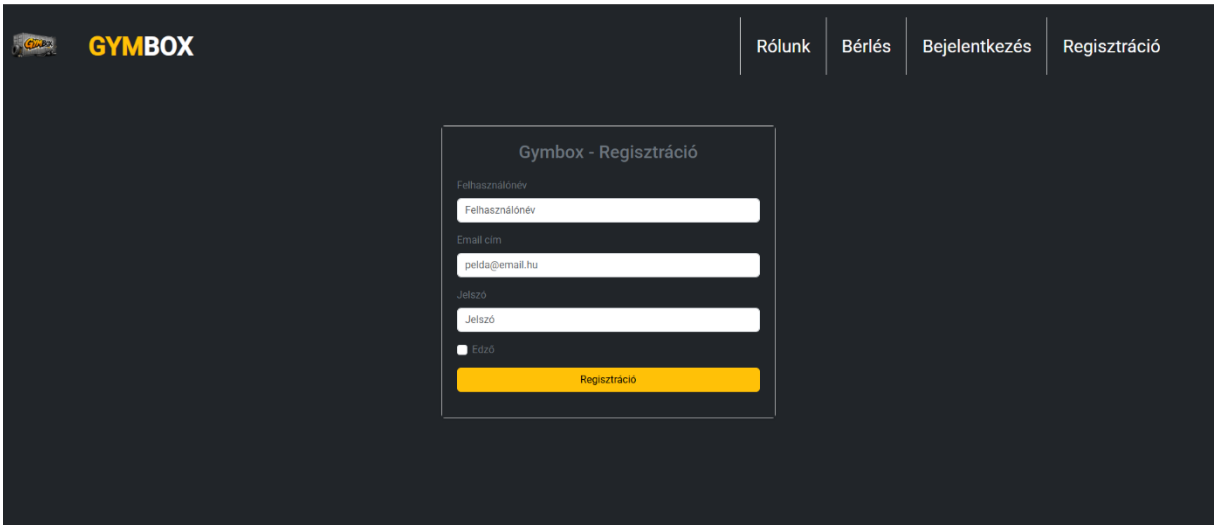
7. Fő oldal edzőkénti regisztáció

Az ezt követő szekcióban a csoport tagjai, tehát az alkalmazás készítőiről látható 3 kártya, képpel és leírással.

8. Fő oldal projekt készítői

Regisztrációs oldal leírása:

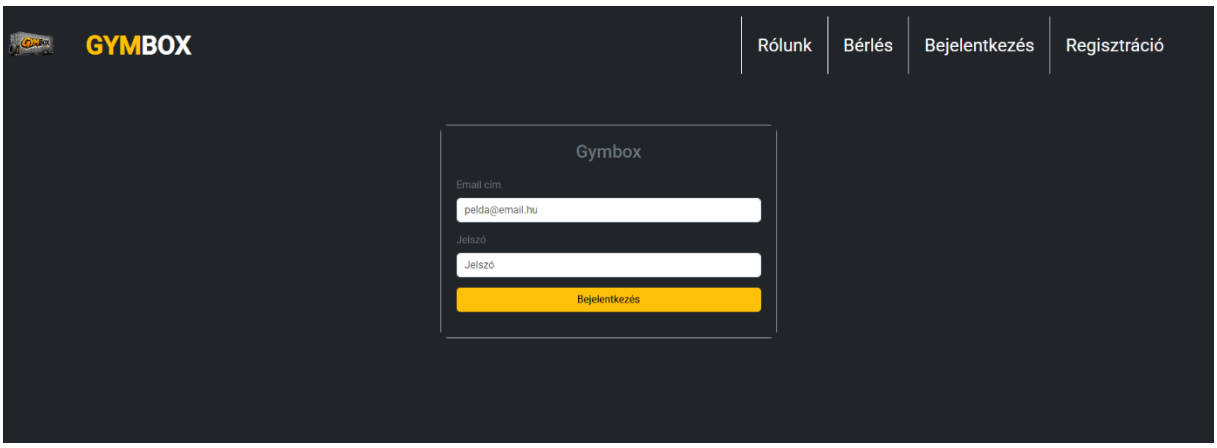
Egy letisztult középre igazított regisztrációs dobozt láthatunk. Amelyben felhasználónevet, e-mail címet és jelszót kérünk be. Valamint egy pipálható mező is látható, hogy a regisztráló fél edzőként szeretne-e regisztrálni. Ezeket az adatokat Angularban az adott komponens TypeScript fájljában hiba kezeljük, hogy csak akkor tudjanak regisztrálni, ha a feltételek megvalósulnak (kötelező mezők kitöltése).



9. Regisztrációs oldal

Bejelentkezés oldal leírása:

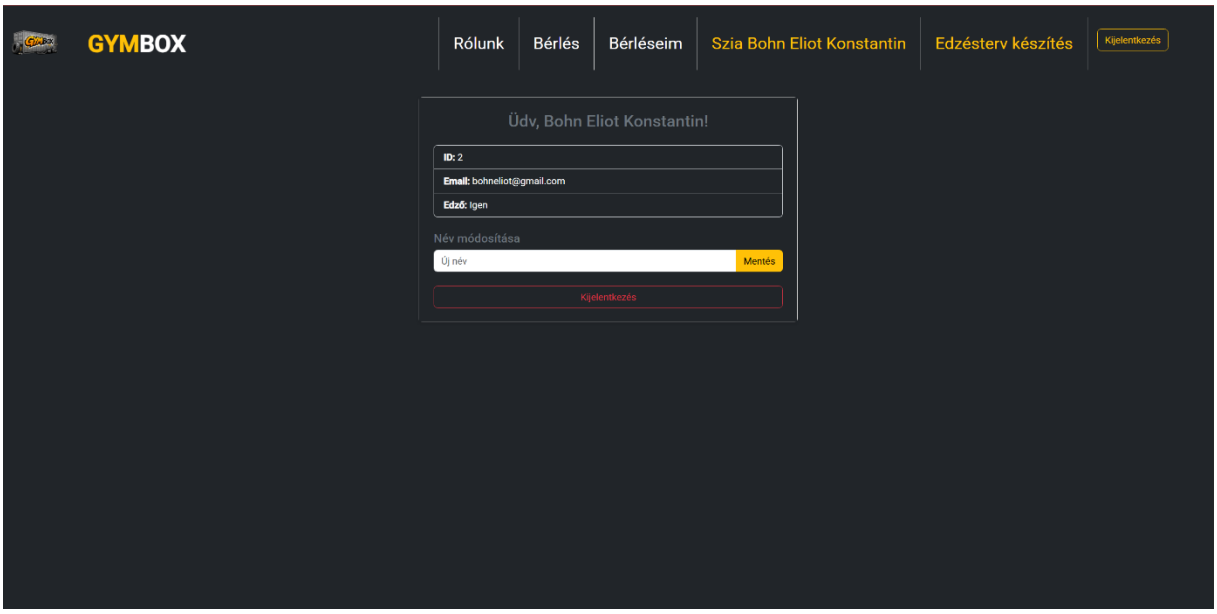
A bejelentkezés oldal a regisztrációs oldal mintájára készült, annyi különbséggel, hogy a regisztrációs oldalon megadott adatokat eltároltuk az adatbázisban. Így a felhasználót létrehoztuk és csak egy e-mail és jelszót kérünk a belépéshez. A bejelentkezést a backendben a logIn metódussal vizsgáljuk.



10. Bejelentkezés oldal

Felhasználó oldal leírása:

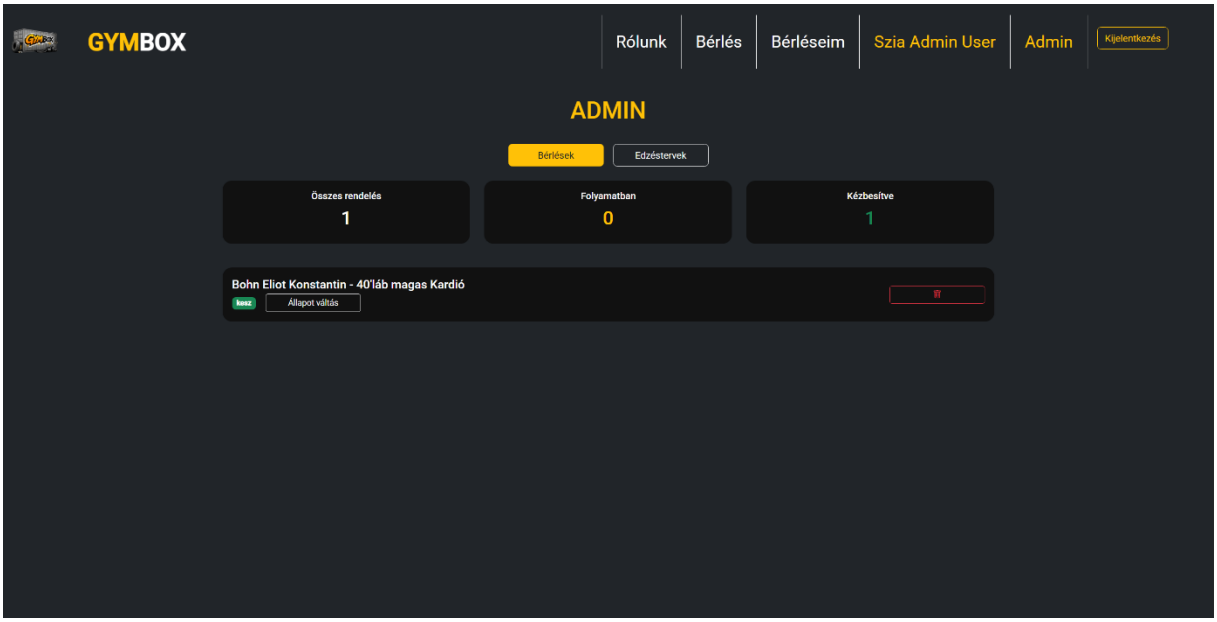
A bejelentkezést követően az eltárolt felhasználó egy külön felhasználó részt kap a navigációs sávban. Ez egy letisztult oldal, középre zárt dobozban adatokkal ellátva: Id, e-mail és hogy a felhasználó edzőként regisztrált-e, valamint lehetőség van a felhasználónév módosítására és a kijelentkezésre.



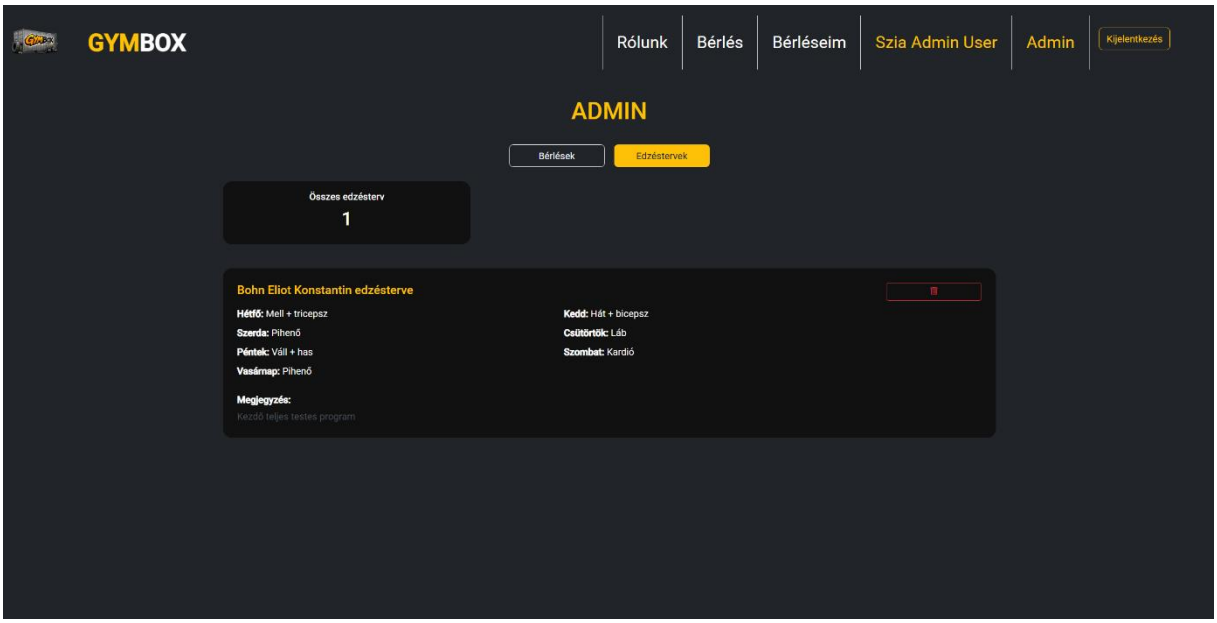
11. Felhasználó oldal

Admin oldal leírás:

Ez egy speciális jogú felhasználó. Ezzel a jogkörrel való bejelentkezés esetén egy újabb felület jelenik meg a navigációs sávban. Erre kattintva megtekinthetjük az összes rendelést és annak státuszát. Valamint az összes edzéstervet és ezek tartalmát.



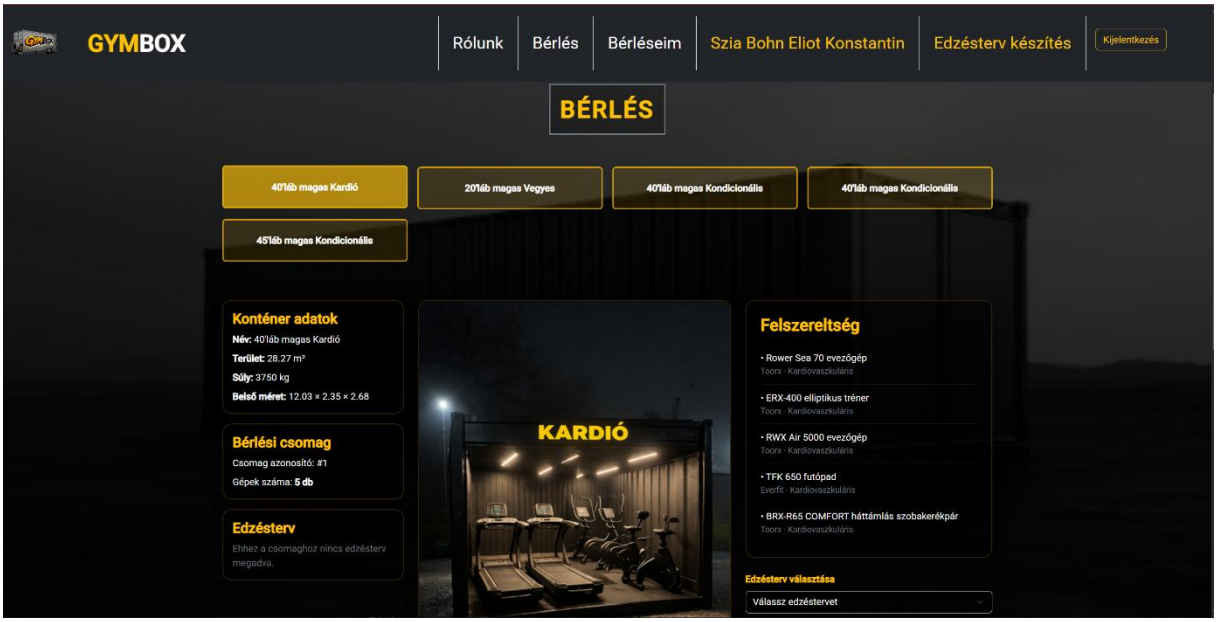
12. Bérlések megtekintése



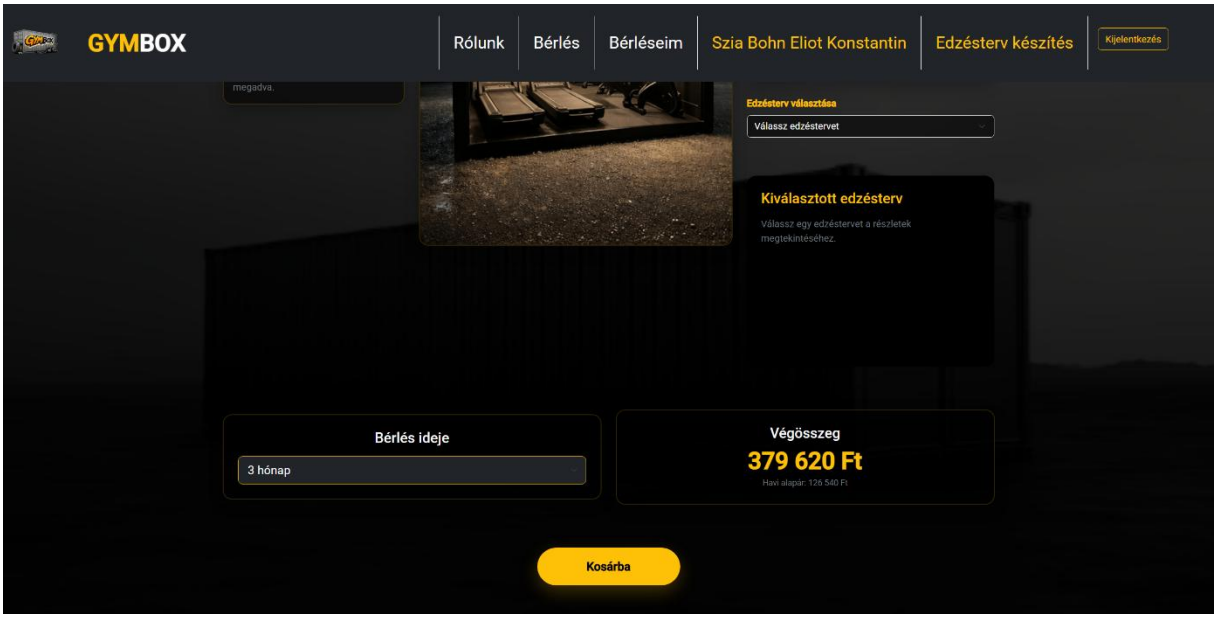
13. Edzéstervek megtekintése

Bérlés oldal leírása:

Az oldalon megtalálható az összes kínált konténer és annak minden adata. A konténer adatai, a bérlés csomag adatai és a konténer felszereltsége jelenik meg. Valamint az a lehetőség, hogy választunk-e hozzá edzéstervet. Az oldal alján a bérlés idejét választhatjuk ki, mellette pedig látható a bérlés ára. A kosárba gombra kattintva kosárba helyezzük, amit a kosár oldalon tárolunk.



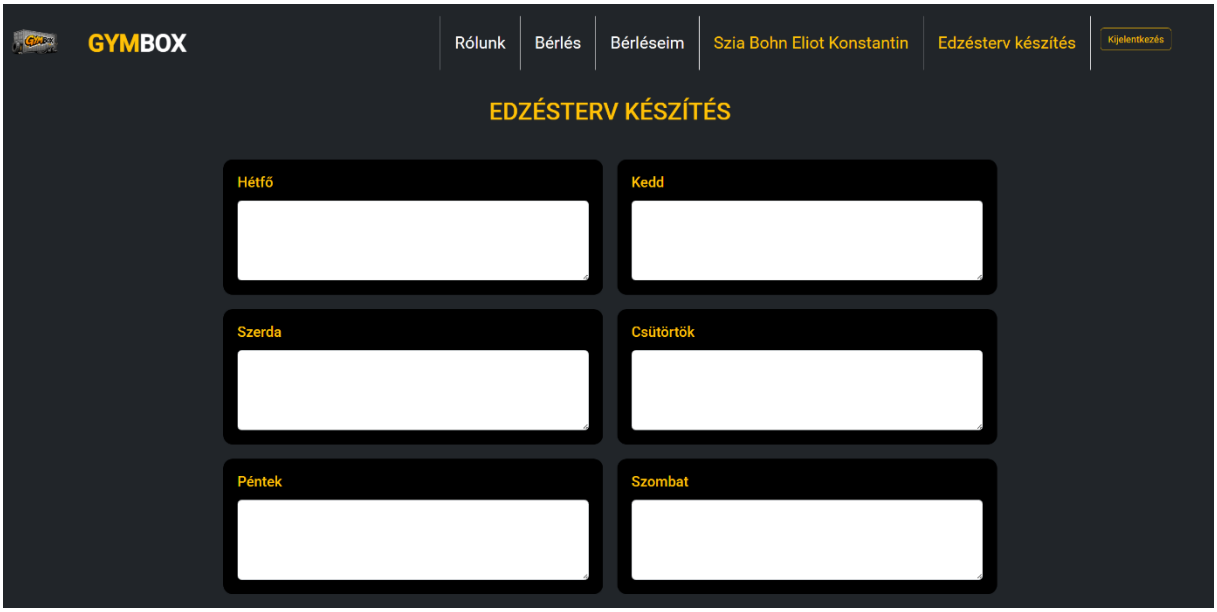
14. Bérlési adatok



15. Bérlési adatok összesítése

Edzésterv készítés oldal leírása:

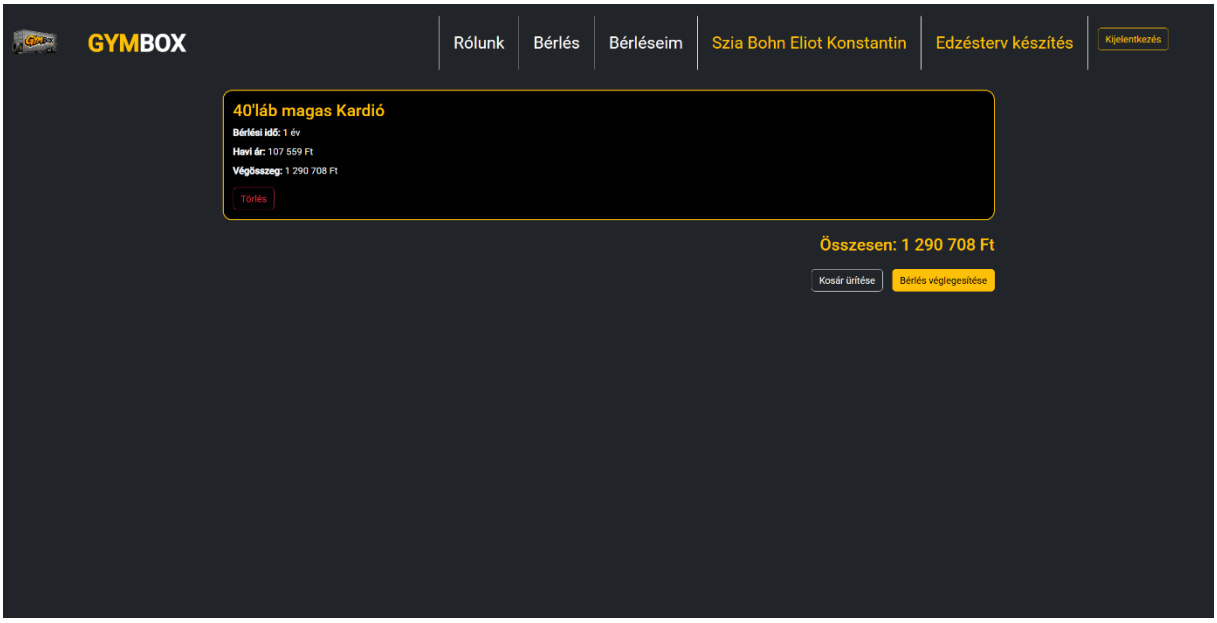
Ez az oldal csak akkor jelenik meg, ha a felhasználó edzőként regisztrált. Itt az oldalon napokra bontott edzéstervet írhat és csatolhat az edző a konténerekhez. Mindegyik naphoz tartozik egy szövegdoboz, amelybe be lehet gépelni az adott edzéstervet.



16. Edzésterv készítés oldal

Kosár oldal leírása:

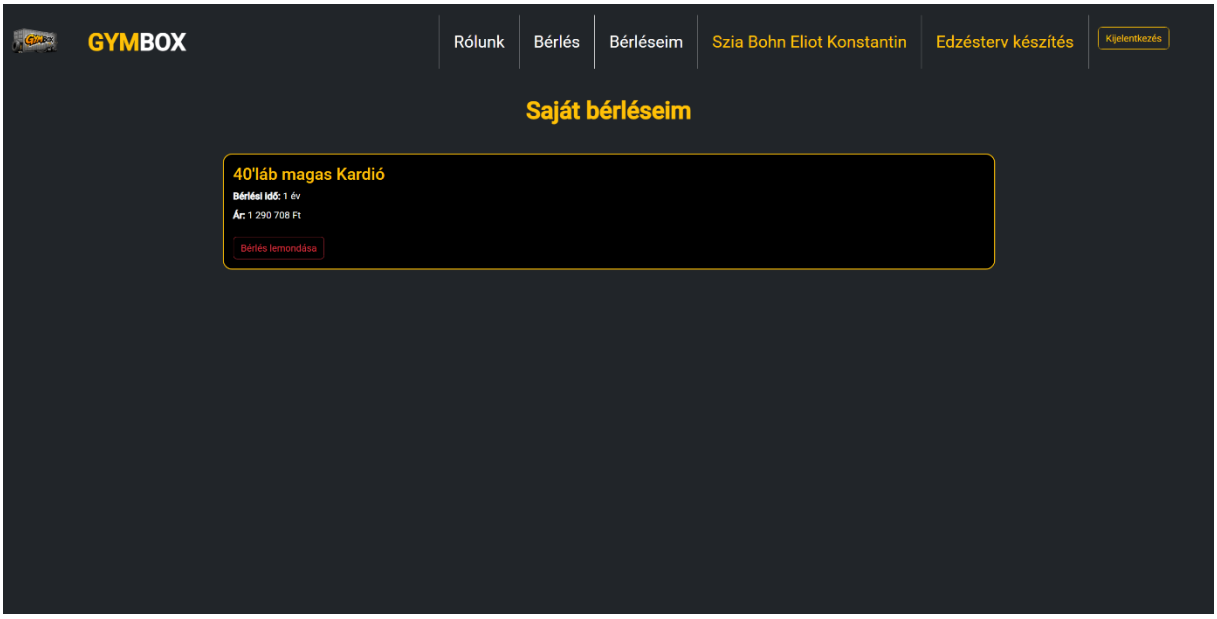
Ha a bérlés oldalon rákattintottunk a „kosárba” gombra akkor ezen az oldalon megjelenik a bérlés. Itt adatokat találhatunk a rendelésről. Bérlés időt, havi árat és a végösszeget is megjeleníti. Lehetőség van a törlésére, a kosár ürítésére vagy a rendelés véglegesítésére.



17. Kosár oldal

Saját bérléseim oldal leírása:

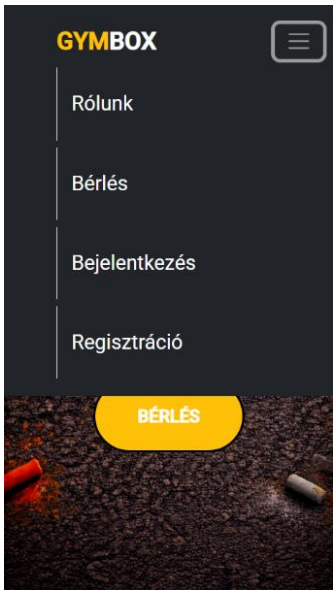
Ha egy bérlést leadtunk, megrendeltünk egy konténert, akkor a navigációs sávon a „bérléseim” fülre kattintva ez az oldal fogad minket. Itt láthatunk egy középre igazított dobozban a bérlésről adatokat: a bérlés idejét és árát. Lehetőség van lemondani a bérlést.



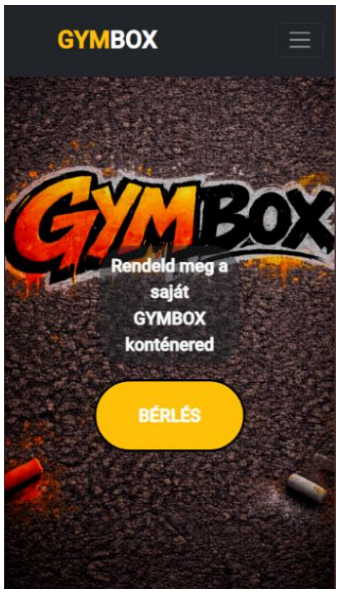
18. Bérléseim oldal

Reszponzivitás:

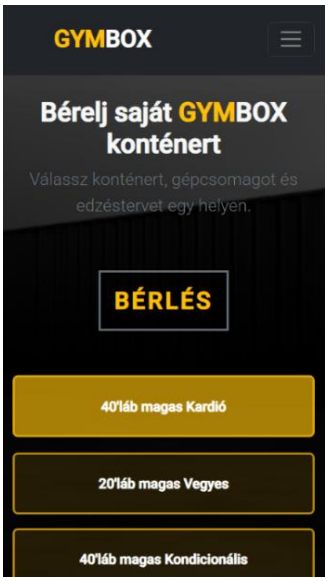
A teljes webalkalmazás reszponzív eszközök széles skálájára. Ez abban nyilvánul meg, hogy az egyes kártyák, amelyek gépes nézetben egymás mellett vannak telefonos nézetben egymás alá esnek. A navigációs sáv ahelyett, hogy egy teljesen nyitott fejléc maradna, egy kattintásra egymásalá lenyíló menüvé válik. Ezzel is képviseli a Gymbox a modern követelményeket és az előrehaladást.



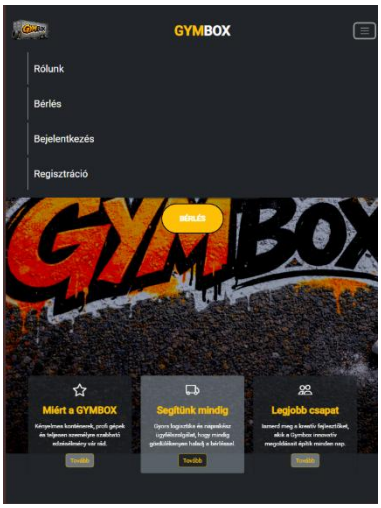
19.Reszponzív navigációs sáv



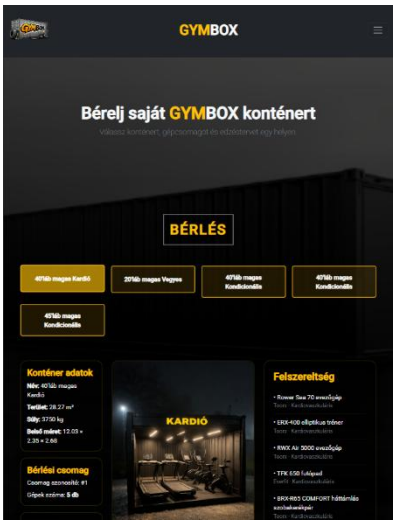
20.Reszponzív főoldal



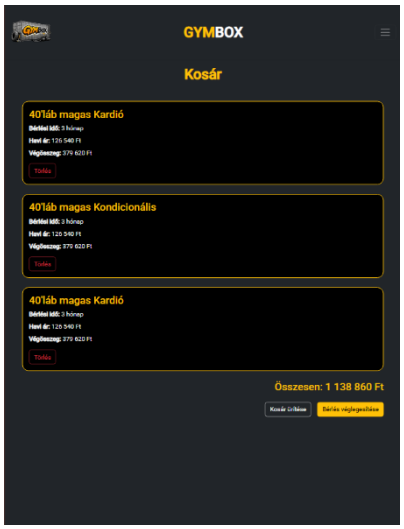
21. Reszponzív bérlés oldal



22. Reszponzív főoldal tablet nézetben



23. Reszponzív bérlés oldal tablet nézetben



24. Reszponzív kosár oldal tablet nézetben

Fejlesztői dokumentáció:

Alkalmazott fejlesztői és csoportmunka eszközök:

- Discord: Kommunikáció a projekt összerakásában
- Trello: Feladatok, feladat fázisok felosztása a csapattagok számára
- GitHub: A projekt tárolása és verziómenedzselése

Adatbázis:

- MySQL: Itt tároljuk az adatbázist
- Xampp: apache szerver futtatása, SQL kód tesztelése lekérdezésekkel
- Visual Studio Code: SQL kód megírása

Backend:

- Visual Studio Code: Laravel api és kontrollerek megírása
- Thunder Client (Visual Stuido Code extension): Kontrollerek tesztelése
- Xampp: adatbázis lokális futtatása és kezelése
- Laravel: php alapú keretrendszer az apikhoz

Frontend:

- Visual Studio Code: a weboldalak kódjának alap szerkezetének megírása, szerkesztése
- Angular: az alap keretrendszer a webalkalmazáshoz
- Figma: az oldalak kezdeti tervei

Adatbázis

Adatbázis modell és leírás:

A GymBox alkalmazás adatbázisa több, egymással kapcsolatban álló adattáblából épül fel. Az adatbázis célja, hogy a felhasználók, konténerek, gépek, gépcsomagok és bérlések adatait strukturáltan, összekapcsoltan és biztonságosan tárolja.

Konténerek tábla:

A különböző méretű és típusú edzőkonténerek adatait tartalmazza. A táblában tároljuk a súlyt, méreteket és a hasznos alapterületet, amelyet négyzetméterben eltárolunk.

id	int	egyedi azonosító és elsődleges kulcs
kontenerNev	varchar, 255 karakter hosszú	konténer típusa
sulyKG	int	konténer saját tömege
belSzelesseg	double	konténer belső szélessége
kulSzelesseg	double	konténer külső szélessége
belMagassag	double	konténer belső magassága
belHosszusag	double	konténer belső hosszúsága
kulMagassag	double	konténer külső magassága
kulHosszusag	double	konténer külső hosszúsága
negyzetMeter	double	konténer hasznos alapterülete

Felhasználó tábla:

A felhasználók alapadatait tartalmazza. Két idegen kulccsal rendelkezik: egy értékeléshez és egy konténerhez kapcsolódik.

id	int	egyedi azonosító és elsődleges kulcs
nev	varchar, 255 karakter hosszú	felhasználó neve
email	varchar, 255 karakter hosszú	felhasználó e-mail címe
jelszo	varchar, 255 karakter hosszú	felhasználó jelszava
edzoE	bool	felhasználó jogköre
ertekeles_id	int	idegen kulcs, hivatkozás az ertekeles táblára
kontener_id	int	idegen kulcs, hivatkozás a kontenerek táblára

Gépek tábla:

A konténerekben elhelyezett edzőgépek adatait tartalmazza. A gépeknél tároljuk a típust, márkát, árat és méreteket.

id	int	egyedi azonosító és elsődleges kulcs
nev	varchar, 255 karakter hosszú	gép neve
ar	int	a gép ára
marka	varchar, 255 karakter hosszú	a gép gyártója
kategoria	varchar, 255 karakter hosszú	a gép kategóriája (kondícionális/kardiovaszkuláris)
gepSuly	int	a gép össztömege
gepHossz	double	a gép hossza
gepSzelesseg	double	a gép szélessége
gepMagasasg	double	a gép magassága

Gépcsomag tábla:

A gépcsomagok tárolják, hogy mely gépek kerülnek egy konténerbe. Öt gép azonosítóját tartalmazza, mindegyik idegen kulccsal kapcsolódik a gépek táblához.

id	int	egyedi azonosíót és elsődleges kulcs
gepId1	int	idegen kulcs, a gepek táblára hivatkozik
gepId2	int	idegen kulcs, a gepek táblára hivatkozik
gepId3	int	idegen kulcs, a gepek táblára hivatkozik
gepId4	int	idegen kulcs, a gepek táblára hivatkozik
gepId5	int	idegen kulcs, a gepek táblára hivatkozik

Edzéstervek tábla:
Ez a tábla egy összekapcsolóként szolgál az edzői felület és a konténerekhez való edzésterv összeállításához.

id	int	egyedi azonosító, elsődleges kulcs
felhasznalo_id	int	idegen kulcs, a felhasznalok táblára hivatkozik
megjegyzes	string	az edzésterv mellé beszúrt megjegyzés, kihagyható
hetfo	string	edzéstervre vonatkozó egyik nap
kedd	string	edzéstervre vonatkozó egyik nap
szerda	string	edzéstervre vonatkozó egyik nap
csutortok	string	edzéstervre vonatkozó egyik nap
pentek	string	edzéstervre vonatkozó egyik nap
szombat	string	edzéstervre vonatkozó egyik nap
vasarnap	string	edzéstervre vonatkozó egyik nap

Csomagok tábla:

Ez a tábla kapcsolja össze a konténerméreteket a gépcsomagokkal és az értékelésekkel. A backend a csomagok alapján építi fel a bérlések választható konfigurációját.

id	int	egyedi azonosítót és elsődleges kulcs
kontener_id	int	idegen kulcs, a kontenerek táblára hivatkozik
gepcsomag_id	int	idegen kulcs, a gepcsomag táblára hivatkozik
ertekeles_id	int	idegen kulcs, az ertekeles táblára hivatkozik
edzesterv_id	int	idegen kulcs, az edzestervek táblára hivatkozik

Bérlés tábla:

A felhasználó által bérelt csomagok adatait tartalmazza. Tárolja a bérlés hosszát és árát is.

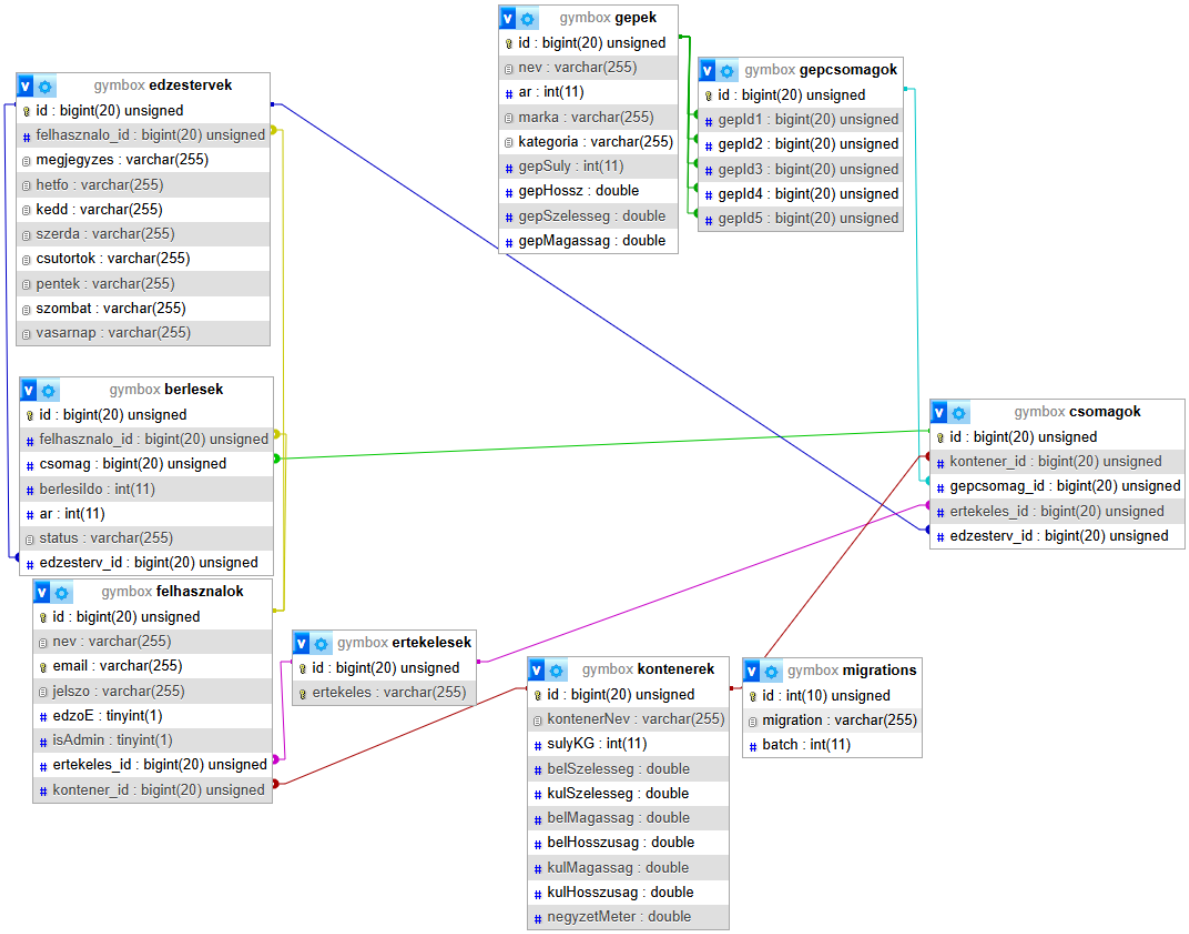
id	int	egyedi azonosító és elsődleges kulcs
csomag	int	idegen kulcs, a csomagok táblára hivatkozik
berlesIdo	int	a szolgáltatás bérlésének ideje
ar	int	a bérlés ára

Kapcsolatok:

Az adatbázis több idegen kulcsos kapcsolatot alkalmaz, amelyek biztosítják az adatok összefüggését:

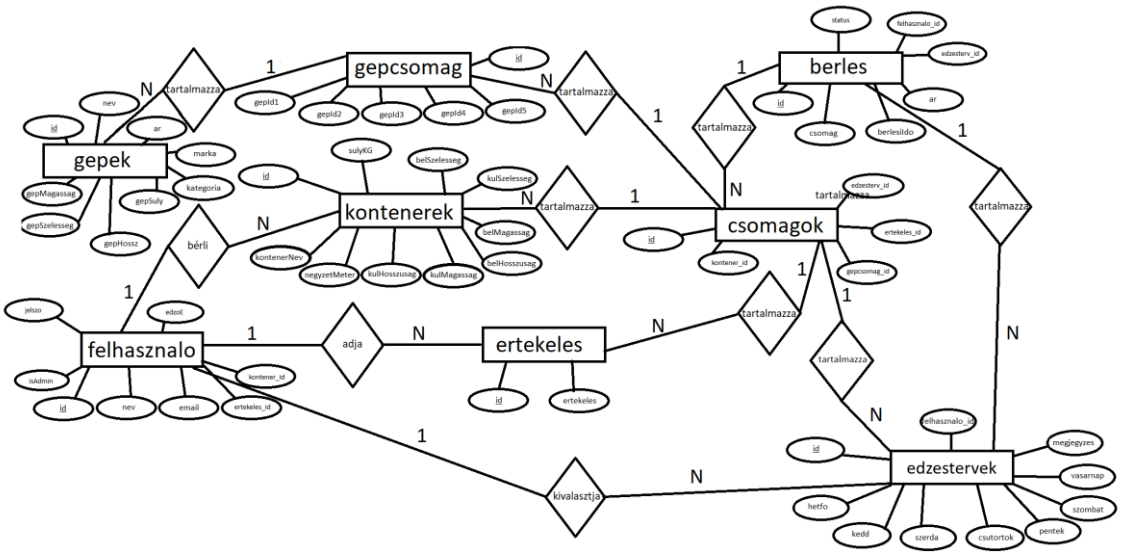
- felhasznalo → ertekeles (1: N)
- felhasznalo → kontenerek (1: N)
- gepcsomag → gepek (több gép egy csomagban)
- csomagok → kontenerek, gepcsomag, ertekeles
- berles → csomagok (1: N)

Kapcsolat ábrák:



25. Táblák közötti kapcsolat ábra

EK modell



26. Egyed - kapcsolat model

Backend:

A GymBox webalkalmazás backendje felelős az üzleti logika megvalósításáért, az adatbázissal való kommunikációért, valamint a frontend kiszolgálásáért. A backend REST API elven működik, amely HTTP kérések segítségével biztosítja az adatok lekérését, létrehozását, módosítását és törlését.

A backend biztosítja:

- az adatok biztonságos kezelését
- az adatbázis táblák közötti kapcsolatok kezelését
- a bérlési folyamat működését
- a felhasználók és csomagok kezelését

Alkalmazott technológiák

- Programozási nyelv: PHP
- Keretrendszer: Visual Studio Code
- Adatbázis-kezelő: MySQL
- API kommunikáció: REST
- Tesztelés: Thunder Client

HTTP kérések:

- /GET
    - Lekérdezi az adatbázis adatait

```
public function index(){
    return Ertekeles::all();
}
```

27.. Get lekérdezés az értékelés kontrollerben

- /POST
    - Új adatot tölt fel az adatbázisba miután ellenőrizte az adatok helyességét

```
public function store(Request $request)
{
            $validator=Validator::make($request->all(),
            [
                "nev"=>"required",
                "email"=>"required",
                "jelszo"=>"required",
                "edzoE"=>"required"
            ]
            );

            if($validator->fails())
            {
                return response()->json(["Hiba üzenet"=>"Hiányzik egy vagy több kötelező adat"],404);
            }
            $felhasznalo=Felhasznalo::create($request->all());
            return response()->json("Sikeres feltöltés!",200);
}
```

28. Post metódus a felhasználó kontrollerben

- /PUT
    - Meglévő adatok módosítása az adatbázisban

```
41
        1 reference | 0 overrides
42      public function update(Request $request, $id): JsonResponse{
43          $felhasznalo=Felhasznalo::find(id: $id);
44          if (is_null(value: $felhasznalo)){
45              return response()->json(data: ["Hiba!"=> "Nem található felhasználó id!"],status: 404);
46          }
47           $validator=Validator::make(data: $request->all(),
48                  rules: [
49                      "nev"=>"required",
50                      "email"=>"required",
51                      "jelszo"=>"required",
52                      "edzoE"=>"required"
53                  ]
54                  );
55          if($validator->fails()){
56              return response()->json(data: ["Hiba!"=> "Hiányzik egy vagy több kötelező adat"],status: 404);
57          }
58          $felhasznalo->update(attributes: $request->all());
59          return response()->json(data: "Sikeres frissítés!",status: 200);
60
61      }
```

29. Put metódus a felhasználó kontrollerben

- /DELETE
    - Meglévő adatokat töröl az adatbázisból

```
public function destroy($id)
{
    $felhasznalo=Felhasznalo::find($id);
    if (is_null($felhasznalo)) {
        return response()->json(["Hiba!"=>"Nem található felhasználó id!"],404);
    }

    $felhasznalo->delete();
    return response("Sikeres törlés",200);
}
```

30. Delete metódus a felhasználó kontrollerben

- GetById
    - Lekérdezi az adatot a keresett Id alapján

```
public function getById($id){
    $ertekeles=Ertekeles::Find($id);
    if (is_null($ertekeles)) {
        return response()->json(["Hiba!"=>"Nem található értékelés id!"],404);
    }

    return response($ertekeles,200);
}
```

31. GetById lekérdezés az értékelés kontrollerben

- GetMoreThan
    - Lekérdezi az adatot egy reláció segítségével. Lekérésnél megadunk egy adatot és megvizsgálja, hogy van-e annál magasabb érték, ha van akkor azt az adatot vagy adatokat kapjuk.

```
public function getBiggerArea($negyzetMeter){
    $kontener=Kontener::where("negyzetMeter",'>',$negyzetMeter);
    if ($kontener->exists()) {
        return response($kontener->get());
    }
    else{
        return response()->json(["Hiba!"=>"Nem található nagyobb négyzetméterű konténer!"],404);
    }
}
```

32. GetMoreThan lekérdezés a konténer kontrollerben

- GetLessThan
    - A GetMoreThan ellentétje, tehát a megadott adatnál alacsonyabb értéket keres és azt az adatot vagy adatokat adja vissza

```
public function getSmallerArea($negyzetMeter){
$kontener=Kontener::where("negyzetMeter",'<',$negyzetMeter);
if ($kontener->exists()) {
    return response($kontener->get());
}
else{
    return response()->json(["Hiba!"=>"Nem található kisebb négyzetméterű konténer!"],404);
}
```

33. GetLessThan lekérdezés a konténer kontrollerben

- FilterBy
    - Lekérdezi az adatokat egy reláció segítségével. Lekérésnél megadunk egy adatot és megvizsgálja, hogy van-e vele egyenlő, ha van akkor azt az adatot vagy adatokat kapjuk vissza

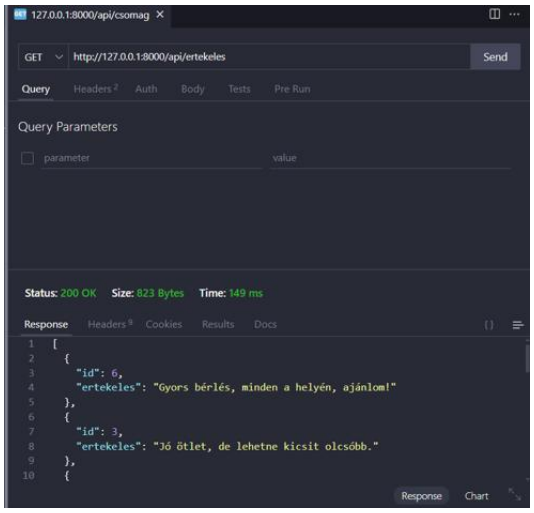
```
public function filterByCateg($kategoria)
{
    $gep=Gep::where("kategoria","=",$kategoria);
    if ($gep->exists()) {
        return response($gep->get());
    }
    else{
        return response()->json(["Hiba!"=>"Nem található a keresett kategória!"],404);
    }
}
```

32. FilterBy szűrés a gépek kontrollerben

Tesztelések:

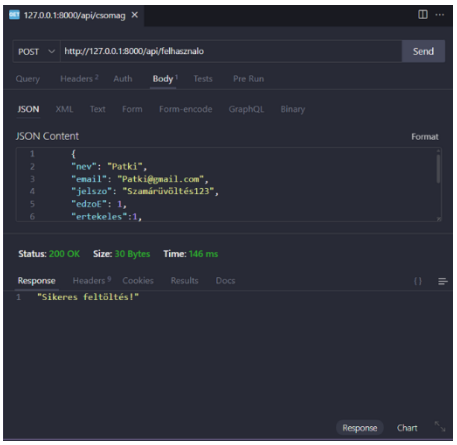
Ezeket a HTTP kéréseket leteszteltük és megnéztük, hogy hogyan válaszol bizonyos feltételekre és milyen hiba üzenetet ad vissza

- Get teszt
- Sikeres teszt



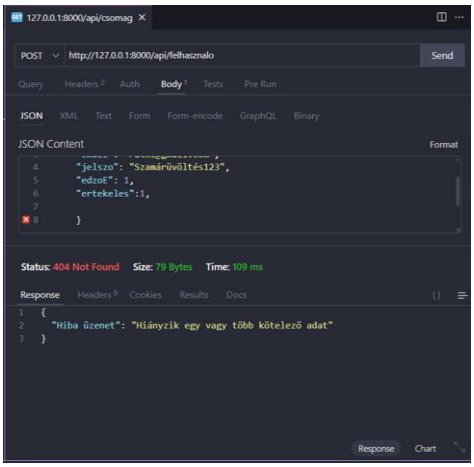
32. Sikeres Get kérés az értékelés Api-ról

- Post teszt
- Sikeres teszt



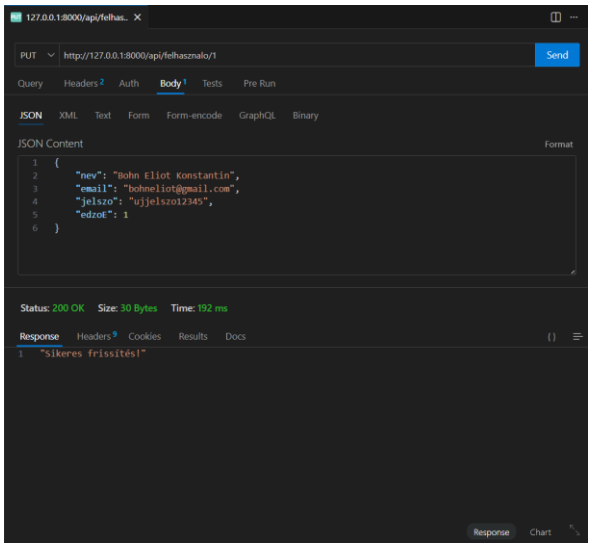
33. Sikeres Post teszt a felhasználó Api-ról

- Sikertelen teszt



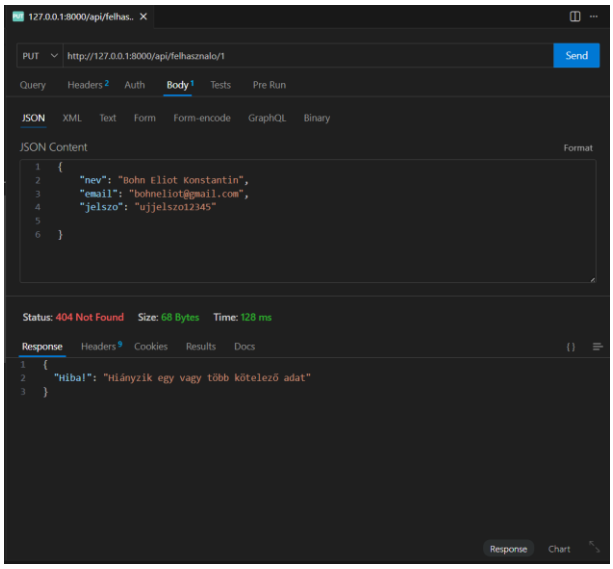
34. Sikertelen Post teszt a felhasználó Api-ról

- Put teszt
- Sikeres teszt



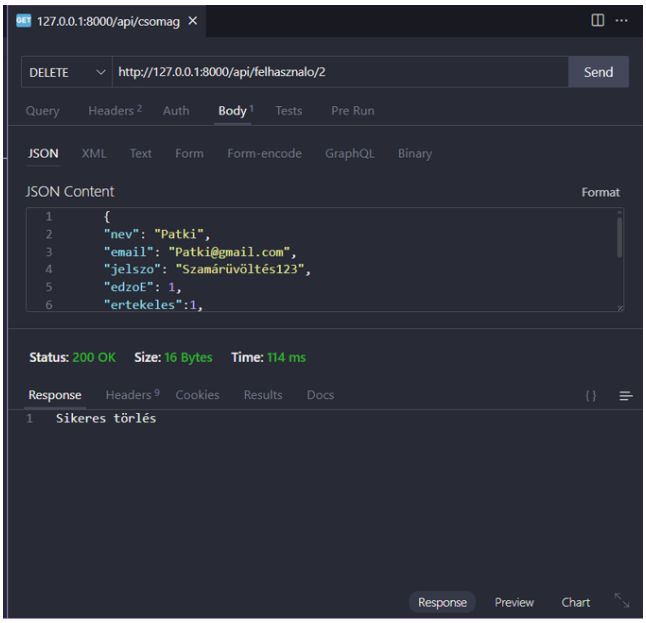
35.. Sikeres Put teszt a felhasználó Api-ról

- Sikertelen teszt



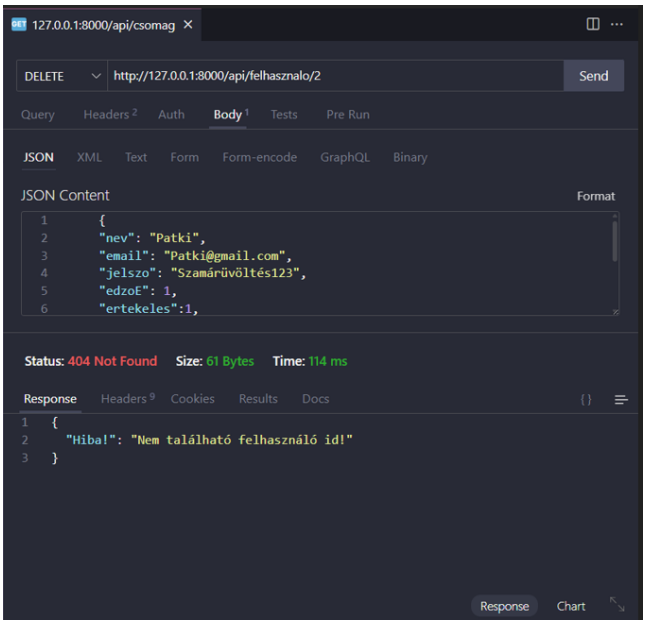
36. Sikertelen Put teszt a felhasználó Api-ról

- Delete teszt
- Sikeres teszt



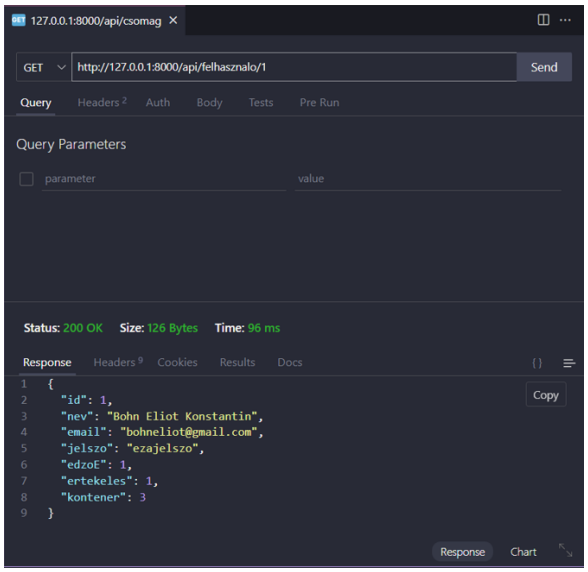
34. Sikeres Delete teszt a felhasználó Api-ról

- Sikertelen teszt



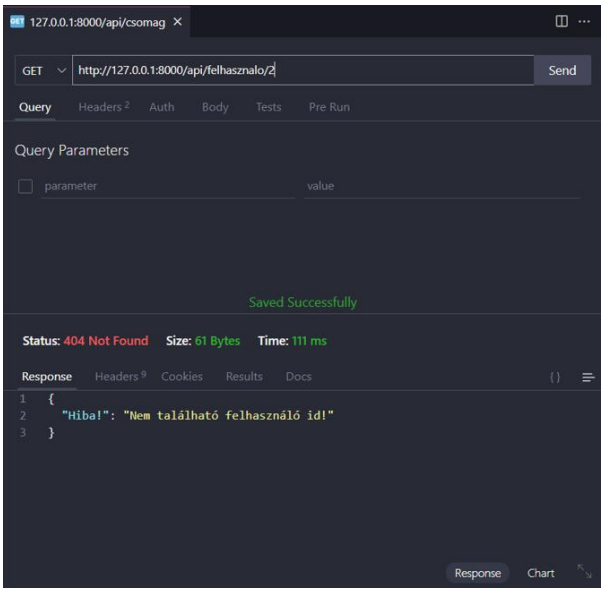
35. Sikertelen Delete teszt a felhasználó Api-ról

- GetById teszt
- Sikeres teszt



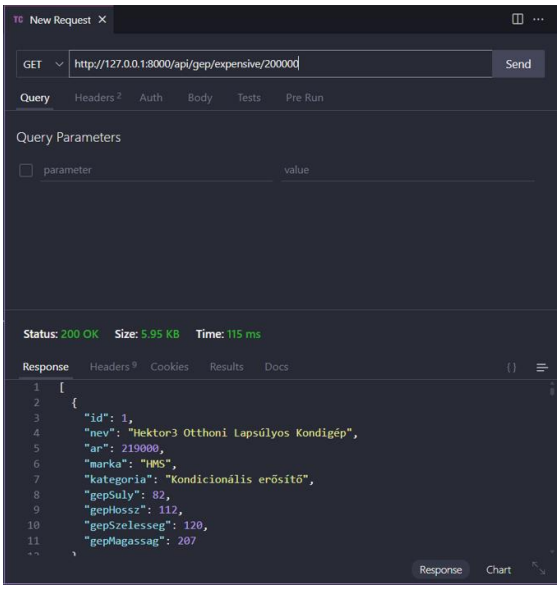
36. Sikeres GetById teszt a felhasználó Api-ról

- Sikertelen teszt



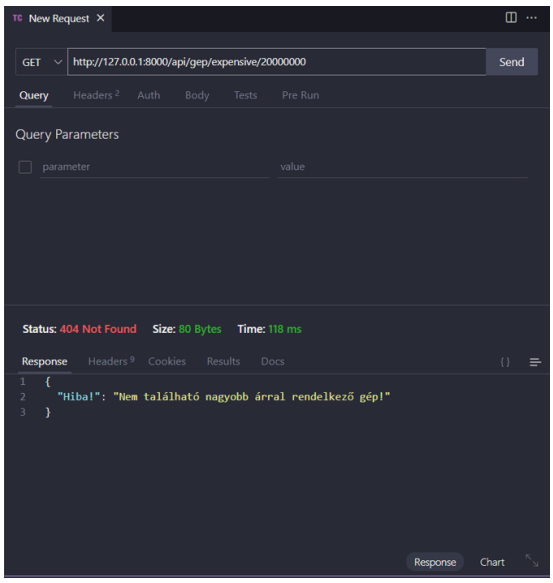
37.Sikertelen GetById teszt a felhasználó Api-ról

- GetMoreThan teszt
- Sikeres teszt



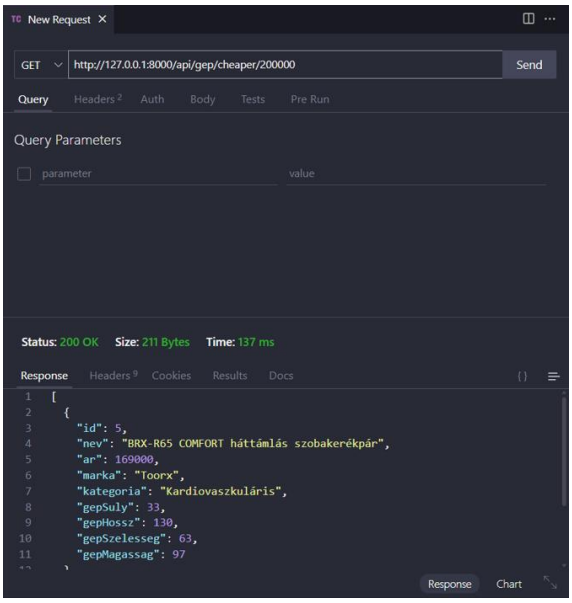
38. Sikeres GetMoreThan teszt a gép Api-ról

- Sikertelen teszt



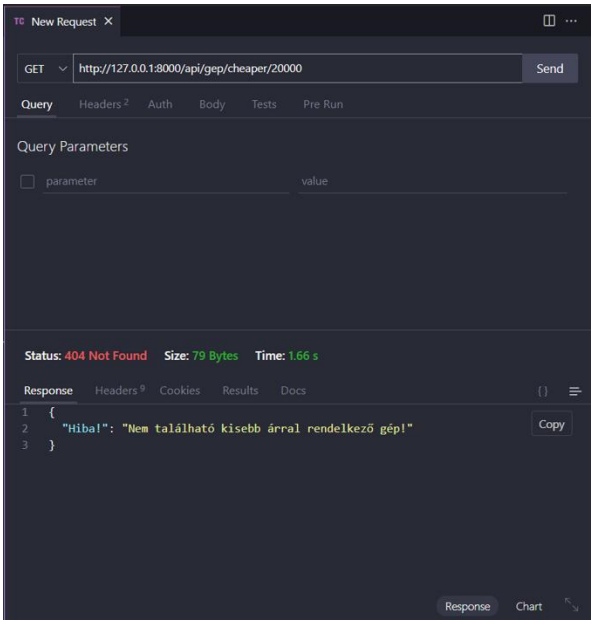
39. Sikertelen GetById teszt a gép Api-ról

- GetLessThan teszt
- Sikeres teszt



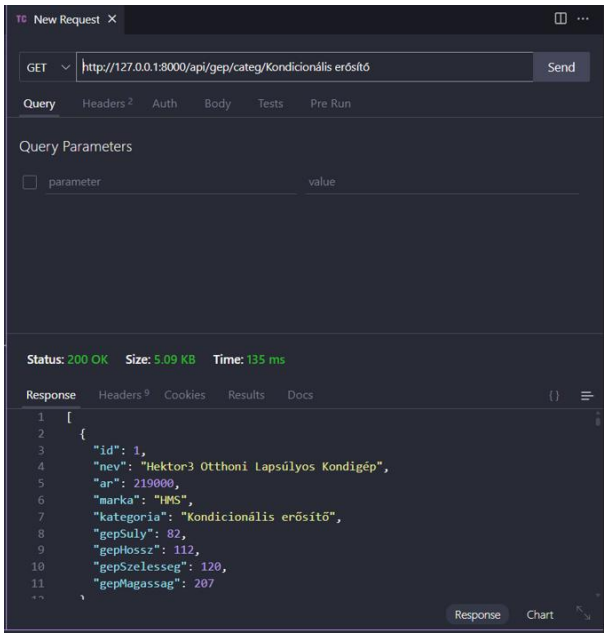
40.Sikeres GetLessThan teszt a gép Api-ról

- Sikertelen teszt



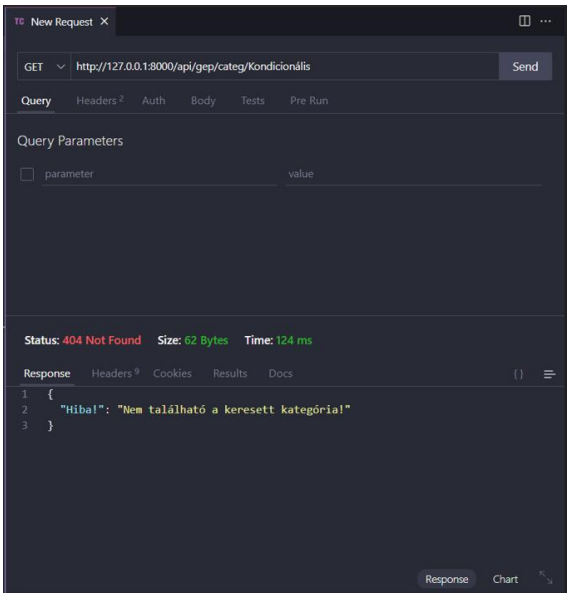
41. Sikertelen GetLessThan teszt a gép Api-ról

- FilterBy teszt
- Sikeres teszt



42. Sikeres FilterBy teszt a gép Api-ról

- Sikertelen teszt



43. Sikertelen FilterBy teszt a gép Api-ról

Frontend:

A frontend felelős a Gymbox felhasználói felületének megteremtéséért. A backenddel való összeköttetés következtében kommunikál az adatbázissal és egy dinamikus webalkalmazásnak tekinthető ennek köszönhetően. A Gymbox egy olyan webes alkalmazás, amely egy innovatív szolgáltatást támogat: mobil edzőtermi konténerek bérlését. A rendszer lényege, hogy a felhasználók előre összeállított, különböző edzéstípusokra optimalizált konténereket (pl. kardió vagy súlyzós) tudnak kiválasztani és kibérelni.

A Gymbox frontend egy Angular alapú Single Page Application (SPA), amely:

- komponens-alapú felépítést használ, ezáltal minden funkció külön komponens az átláthatóság, könnyű jövőbeli bővítés érdekében
- routing segítségével kezeli a nézeteket
- guard-okkal védi az egyes útvonalakat
- Laravel backenddel kommunikál (feltételezhetően API-n keresztül)

Az Angular alapú frontend feladata, hogy:

- biztosítsa a felhasználóbarát felületet
- kezelje a felhasználói interakciókat
- kommunikáljon a Laravel backend API-val
- dinamikusan jelenítse meg az adatokat (csomagok, konténerek, bérlések)
- ez egy Single Page Application (SPA), ahol az oldalak újratöltés nélkül váltanak

A rendszer központi elemei:

- Root komponens (App)
- Router (útvonalkezelés)
- Feature komponensek (pl. bérlés, kosár, admin)
- Guard-ok (auth, admin, edző)

Felhasználói felület:

- Bootstrap alapú design
- reszponzív megjelenés

- fix navigációs sáv, amely egy külön komponens a gyakori felhasználás és egység megőrzése következtében
- dinamikus tartalom betöltés

Frontendünk egy modern, jól strukturált Angular alkalmazás, amely: egyedi üzletimodellt támogat (mobil edzőterem bérlés) teljes bérlési folyamatot lefed több szerepkört kezel (user, admin, edző) biztonságos és skálázható felépítésű. A rendszer célja, hogy a felhasználók gyorsan és egyszerűen tudjanak komplett edzőtermi konténereket bérelni.

Az alkalmazás routing rendszere az Angular beépített router moduljára támaszkodik, amely lehetővé teszi a navigációt az egyes oldalak között. A lényeg az, hogy a felhasználó által megadott URL alapján az alkalmazás tudja, melyik komponenst kell a képernyőn megjeleníteni. Ezt a működést az app.routes.ts fájlban határozzák meg, ahol az egyes útvonalak (path-ok) és a hozzájuk tartozó komponensek össze vannak kapcsolva.

A központi elem a <router-outlet>, ami az alkalmazás fő komponensének HTML fájljában helyezkedik el. Ez egy helykitöltő elem, ahová az Angular automatikusan betölti a megfelelő komponenst a jelenlegi útvonal alapján. Például, ha a felhasználó a „/login” útvonalra lép, akkor a bejelentkezési komponens fog megjelenni itt.

A navigáció történhet felhasználói interakcióval, például routerLink segítségével, vagy programozott módon a Router szolgáltatás használatával. Így például sikeres bejelentkezés után a felhasználó automatikusan átirányítható az admin felületre.

A routing rendszer lehetőséget biztosít a hozzáférés-szabályozásra is, amit úgynevezett guard-ok révén valósíthatunk meg. Ezek segítségével bizonyos oldalak, mint például az admin felület, csak a megfelelő jogosultsággal rendelkező felhasználók számára érhetők el. Ha valaki nem rendelkezik a szükséges jogosultsággal, a rendszer megakadályozza a navigációt.

Az alkalmazásban dinamikus útvonalak is használhatóak, ahol paraméterek segítségével adatokat lehet átvinni az URL-ben. Például egy felhasználói azonosítót átadhatunk az útvonal részeként, amit a komponens az ActivatedRoute használatával tud kiolvasni és feldolgozni.

Összességében a routing biztosítja az alkalmazás logikus felépítését és a felhasználóbarát navigációt, lehetővé téve, hogy az egyes funkciók különálló komponensekben működjenek, miközben egy egységes rendszer részeként funkcionálnak.

Fontosabb frontend metódusok és hibakezelések:

A Gymbox felület regisztrációjában a megadott mezőket (felhasználónév, e-mail, jelszó) kötelező kitölteni. Ugyanis, ha ezt nem teszi a felhasználó, akkor hibaüzenetet kap és a regisztráció gomb nem kattintható

```
registration() {
if (!this.felhasznalo.nev || !this.felhasznalo.email || !this.felhasznalo.jelszo) {
  this.message = 'Kérem, töltse ki az összes mezőt!';
  return;
}

console.log('registration meghívva');
if (this.isRegistering) return;
this.isRegistering = true;
this.message = '';

this.auth.register(this.felhasznalo.nev, this.felhasznalo.jelszo, this.felhasznalo.email, this.felhasznalo.edzoE)
  .subscribe({
    next: (res) => {
      console.log('register next',res);
      this.message = 'Sikeres regisztráció!';
      this.isRegistering = false;
      this.cdr.detectChanges();

      setTimeout(()=>{
        this.router.navigate(['/loggedin']);
      },2000);

    },
    error: (err) => {
      console.log('register error',err);
      this.message = 'Hiba történt a regisztráció során.';
      this.isRegistering = false;
      this.cdr.detectChanges();
    }
  });
}
```

44. Regisztráció hibakezelése

A bejelentkezési felületet akkor tudjuk használni, ha a regisztráció sikeresen végződött. Ezáltal az adatbázisban eltároljuk a felhasználót és a regisztrációnál megadott fiók adatokkal be tud jelentkezni a felületre. Ha helytelen e-mail címet vagy jelszót adunk meg, akkor hiba üzenetet kapunk.

```
24  loginBtn() {
25    this.auth.getFelhasznalo(this.felhasznaloEmail, this.felhasznaloJelszo).subscribe({
26      next: (felhasznalo) => {
27        this.auth.loggedinFelhasznalo.set(felhasznalo);
28        this.snackbar.success('Sikeres bejelentkezés');
29        this.router.navigate(['/loggedin']);
30        this.auth.isLoggenIn.set(true);
31      },
32      error: (error) => {
33        if (error.status === 401) {
34          this.snackbar.error('Hibás e-mail cím vagy jelszó!');
35        } else {
36          this.snackbar.error('Hiba történt!');
37        }
38      }
39    });
40  }
```

45. Bejelentkezés hibakezelés

A kosár oldalon található „bérlés véglegesítése” gombra kattintva leadjuk a megrendelést és visszajelzést kapunk róla, hogy sikeres vagy sikertelen-e a megrendelés.

```
71      forkJoin(requests).subscribe({
72        next: () => {
73          this.snackbar.success('Sikeres bérlés!');
74          this.kosarService.clearKosar();
75          this.items = [];
76          this.mentesFolyamatban = false;
77          this.cdr.detectChanges();
78        },
79        error: (err) => {
80          console.error(err);
81          this.snackbar.error('Nem sikerült a bérlés mentése.');
82          this.mentesFolyamatban = false;
83          this.cdr.detectChanges();
84        }
85      });
86    }
```

46.Rendelés leadása visszajelzés

A főoldalon található három középre zárt kártyán találhatunk leírást konténereinkről, gépjeinkről és edzőinkről. Itt, ha rákattintunk az egyikre egy oldalt tölt be róluk. A következő képeken erről a három kártyáról kódrészletet, amiben a hibakezelés is benne van.

```
21
22  ngOnInit(): void {
23    this.kontenerService.getAll().subscribe({
24      next: (data) => {
25        this.kontenerek = data;
26        this.kiemeltKontener = data.length ? data[0] : null;
27        this.tobbiKontener = data.slice(1);
28        this.betoltes = false;
29        this.cdr.detectChanges();
30      },
31      error: (err) => {
32        console.error(err);
33        this.hiba = 'Nem sikerült betölteni a konténereket.';
34        this.betoltes = false;
35      }
36    });
37  }
```

47.Konténereink oldal betöltés/hibakezelés

```
ngOnInit(): void {
  this.gepService.getAll().subscribe({
    next: (data) => {
      this.gepek = data;
      this.betoltes = false;
      this.cdr.detectChanges();
    },
    error: (err) => {
      console.error(err);
      this.hiba = 'Nem sikerült betölteni a gépeket.';
      this.betoltes = false;
    }
  });
}
```

48.Gépeink betöltése/hibakezelés

```
19
20  ngOnInit(): void {
21    this.http.get<any[]>(`${API_URL}/edzok-edzestervek`).subscribe({
22      next: (data) => {
23        this.edzok = data;
24        this.betoltes = false;
25        this.cdr.detectChanges();
26      },
27      error: (err) => {
28        console.error(err);
29        this.hiba = 'Nem sikerült betölteni az edzőket.';
30        this.betoltes = false;
31      }
32    });
33  }
```

49. Edzőink betöltése/hiba kezelés

A bérlések oldal betöltéséhez rendelkeznünk kell egy felhasználói fiókkal. Az alábbi kódrészletben látható, hogy ha nem rendelkezünk hiba üzenetet kapunk: „A bérlések megtekintéséhez be kell jelentkezni.”. Ha van fiókunk akkor betölti az oldalt, de ha valami hiba történne akkor is egy üzenetet kapunk: „Nem sikerült betölteni a bérléseket.”.

```
30
31  ngOnInit(): void {
32    const user = this.auth.loggedinFelhasznalo();
33
34    if (!user) {
35      this.hiba = 'A bérlések megtekintéséhez be kell jelentkezni.';
36      this.betoltes = false;
37      return;
38    }
39
40    this.berlesService.getUserBerlesek(user.id).subscribe({
41      next: (adatok) => {
42        this.berlesek = adatok;
43        this.betoltes = false;
44        this.cdr.detectChanges();
45      },
46      error: (err) => {
47        console.error(err);
48        this.hiba = 'Nem sikerült betölteni a bérléseket.';
49        this.betoltes = false;
50      }
51    });
52  }
```

50. Egyenlet Bérlések betöltése/hibakezelés

A bérléseim oldalon megjelenik előttünk a bérlésünk néhány adattal. A bérlés lemondása gombra kattintva lehetőségünk van lemondani a bérlést, amely az alábbi kódrészletben láthatóan egy üzenetet fog visszaadni a lemondás eredményességéről.

```
64  lemondas(berlesId: number): void {
65    const dialogRef = this.dialog.open(ConfirmDialogKomponens, {
66      width: '400px',
67      data: {
68        title: 'Bérlés lemondása',
69        message: 'Biztosan le szeretnéd mondani ezt a bérlést?'
70      }
71    });
72
73    dialogRef.afterClosed().subscribe((eredmeny) => {
74      if (!eredmeny) {
75        return;
76      }
77
78      this.berlesService.delete(berlesId).subscribe({
79        next: () => {
80          this.berlesek = this.berlesek.filter(b => b.id !== berlesId);
81          this.snackbar.success('A bérlés sikeresen le lett mondva.');
82          this.cdr.detectChanges();
83        },
84        error: (err) => {
85          console.error(err);
86          this.snackbar.error('Nem sikerült lemondani a bérlést.');
87        }
88      });
89    });
90  }
```

51.Bérlés lemondása/hibakezelés

Ha a felhasználó edzőként regisztrál akkor lehetősége van a konténerekhez edzéstervet írnia. A képen látható kódrészletben egy részletes hibakezelés látható arról, ha nem tölti ki az edző megfelelően a mezőket, akkor egy hiba üzenetet kap. Ha minden feltételnek megfelel a kitöltés megkapjuk, hogy „Edzésterv sikeresen mentve”. Ez az edzésterv el lesz tárolva az adatbázisban és a megrendelők megkapják a konténereikhez.

```
52
53  save(): void {
54  if (!this.isValid()) {
55    this.snackbar.error('Kérlek tölts ki minden mezőt!');
56    return;
57  }
58
59    this.edzestervService.create(this.edzesterv).subscribe({
60      next: () => {
61        this.snackbar.success('Edzésterv sikeresen mentve');
62        this.edzesterv = {
63          felhasznalo_id: this.auth.loggedinFelhasznalo()!.id,
64          hetfo: '',
65          kedd: '',
66          szerda: '',
67          csutortok: '',
68          pentek: '',
69          szombat: '',
70          vasarnap: '',
71          megjegyzes: ''
72        };
73      },
74      error: (err) => {
75        console.error(err);
76        this.snackbar.error('Nem sikerült elmenteni az edzéstervet.');
77      }
78    });
79  }
```

52.Edzésterv létrehozás/hibakezelés

Az alábbi kódrészlet az Admin felülethez kapcsolódik. Mikor rákattintottunk a bérlés törlése gombra akkor egy felugró ablak jön elő, amelyben megerősítés szükséges. Ez a véletlen hibák elkerülése érdekében készült.

```
93    confirmBerlesTorles(id: number): void {
94      const dialogRef = this.dialog.open(ConfirmDialogKomponens, {
95        width: '400px',
96        data: {
97          title: 'Bérlés törlése',
98          message: 'Biztosan törölni szeretnéd ezt a bérlést?'
99        }
100     });
101
102     dialogRef.afterClosed().subscribe((eredmeny) => {
103       if (eredmeny) {
104         this.deleteBerles(id);
105       }
106     });
107   }
108
109   confirmEdzestervTorles(id: number): void {
110     const dialogRef = this.dialog.open(ConfirmDialogKomponens, {
111       width: '400px',
112       data: {
113         title: 'Edzésterv törlése',
114         message: 'Biztosan törölni szeretnéd ezt az edzéstervet?'
115       }
116     });
117
118     dialogRef.afterClosed().subscribe((eredmeny) => {
119       if (eredmeny) {
120         this.deleteEdzesterv(id);
121       }
122     });
123   }
```

53.Admin edzésterv/konténer törlés

Az alábbi kódrészlet az Admin felülethez kapcsolódik. Ha adminként jelentkezünk be, akkor lehetőségünk van az összes leadott rendelést, készített edzéstervet megtekinteni. Valamint jogunk van törölni ezeket a bérléseket és edzésterveket.

```
deleteBerles(id: number): void {
  this.berlesService.delete(id).subscribe({
    next: () => {
      this.berlesek = this.berlesek.filter(b => b.id !== id);
      this.snackbar.success('A bérlés sikeresen törölve lett.');
      this.cdr.detectChanges();
    },
    error: (err) => {
      console.error('Hiba bérlés törlésekor:', err);
      this.snackbar.error('Nem sikerült törölni a bérlést.');
    }
  });
}

deleteEdzesterv(id: number): void {
  this.edzestervService.delete(id).subscribe({
    next: () => {
      this.edzestervek = this.edzestervek.filter(e => e.id !== id);
      this.snackbar.success('Az edzésterv sikeresen törölve lett.');
      this.cdr.detectChanges();
    },
    error: (err) => {
      console.error('Hiba edzésterv törlésekor:', err);
      this.snackbar.error('Nem sikerült törölni az edzéstervet.');
    }
  });
}
```

54. Egyenlet Admin edzésterv/bérlés törlés

Adminként akár a rendelés státuszát is tudjuk módosítani folyamatban lévőről kiszállítottra. Itt is található egy hibakezelés, ami egy üzenetet ad vissza, ha sikertelen volt a módosítás: Hiba a státusz váltáskor: Nem sikerült módosítani az állapotot.”.

```
toggleStatus(id: number): void {
  this.berlesService.updateStatus(id).subscribe({
    next: () => {
      this.loadBerlesek();
      this.cdr.detectChanges();
    },
    error: (err) => {
      console.error('Hiba státusz váltáskor:', err);
      this.snackbar.error('Nem sikerült módosítani az állapotot.');
    }
  });
}
```

55. Rendelés státusz módosítása/hibakezelés

Ez a kódrészlet a bérlések oldal betöltéséért felel. A backendben tárolt adatokat megkapja a frontend és ennek köszönhetően megjelennek a feltöltött konténerek minden adatukkal együtt.

```
ngOnInit(): void {
  this.betoltes = true;

  forkJoin({
    csomagok: this.csomagservice.getAll(),
    gepek: this.gepservice.getAll(),
    edzestervek: this.edzestervService.getAll()
  }).subscribe({
    next: ({ csomagok, gepek,edzestervek }) => {
      this.csomagok = csomagok;
      this.gepek = gepek;
       this.edzestervek = edzestervek;

      if (this.csomagok.length > 0) {
        this.selectedCsomag = this.csomagok[0];
      }

      this.betoltes = false;
      this.cdr.detectChanges();
    },
    error: (err) => {
      console.error('API hiba:', err);
      this.hiba = 'Nem sikerült betölteni az adatokat.';
      this.betoltes = false;
      this.cdr.detectChanges();
    },
  });
}
```

56.Bérlések betöltése / hibakezelés

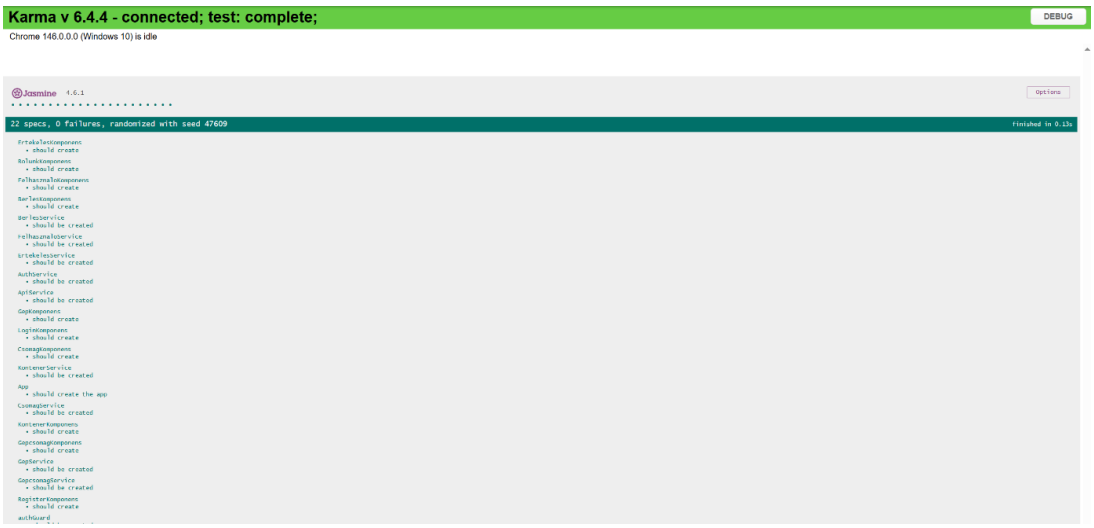
A felhasználónak a navigációs sávban a saját profiljára kattintva van lehetősége a felhasználóneve módosítására. A kódrészletben látható, hogy vannak feltételek, hogy lehessen változtatni a nevet. Ha nem létezik a felhasználó akkor nem lehetséges, ha a név kisebb, mint három karakter nem engedi a változtatást. Ha sikeres vagy sikertelen a felhasználónév váltása akkor mind a két esetben egy üzenetet kapunk róla.

```
23  nevModositas() {
24    const user = this.auth.loggedinFelhasznalo();
25    if (!user) return;
26
27    const nev = this.ujNev.trim();
28    if (nev.length < 3) {
29      this.message = 'A név legyen legalább 3 karakter.';
30      this.cdr.detectChanges();
31      return;
32    }
33
34    this.auth.updateNev(user.id, nev).subscribe({
35      next: () => {
36        this.message = 'Név sikeresen módosítva!';
37        this.ujNev = '';
38        this.cdr.detectChanges();
39      },
40      error: (err) => {
41        console.log('UPDATE ERROR', err);
42        this.message = 'Hiba történt a név módosítása során.';
43        this.cdr.detectChanges();
44      }
45    });
46  }
```

57.Felhasználónév módosítása / hibakezelés

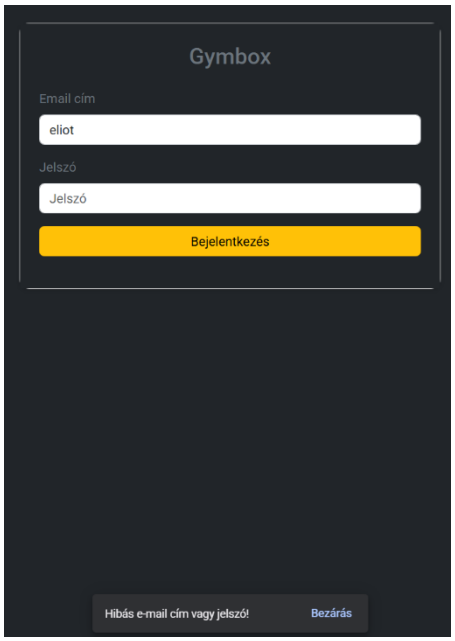
Tesztelés:

Egységtesztet hajtottunk végre a teljes frontenden. Az egységtesztelés során az alkalmazás egyes komponenseit, szolgáltatásait vagy direktíváit izoláltan teszteltük. Ehhez a Jasminet és a Karmát használtuk. A Jasmine egy népszerű JavaScript keretrendszer, amelyet az Angular CLI alapértelmezetten használ. A Karma pedig egy tesztfuttató, amely egy böngészőben hajtja végre a teszteket.

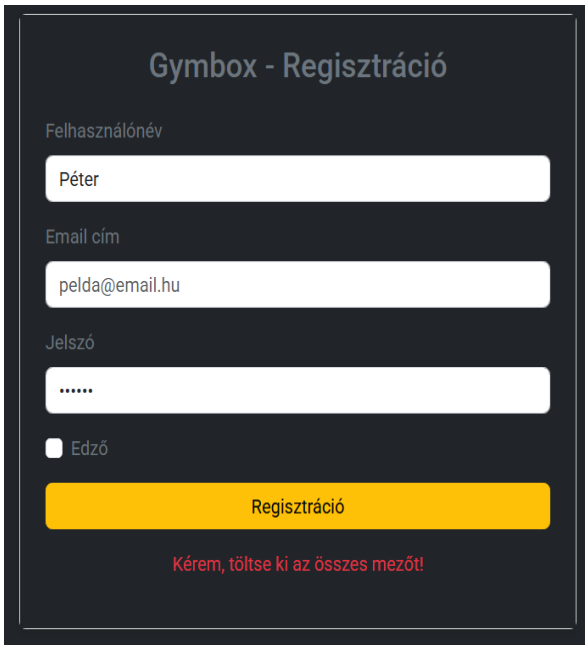


58. Karma webteszt

Valamint oldalunk regisztrációja és bejelentkezése hibakezeléssel van védve. Ennek következtében a megadott feltételek hiányában nem lehet bejelentkezni / regisztrálni. Szintén hibakezeléssel oldottuk meg a bérléseket. Ha a felhasználó nem rendelkezik fiókkal, nem lehet bérelni konténert.



59. Bejelentkezés oldal hibakezelés



60. Regisztráció oldal hibakezelés

Továbbfejlesztési lehetőségek:

A Gymbox webalkalmazás jelenlegi működése lehetővé teszi a felhasználók számára, hogy előre összeállított edzőgép-csomagok közül válasszanak konténereik felszereléséhez. A továbbfejlesztési irányunk célja ennek a modellnek a bővítése egy rugalmasabb, felhasználóközpontú konfigurációs rendszerrel. A jövőben a felhasználók lehetőséget kapnak arra, hogy saját igényeik szerint, egyedi módon állítsák össze a konténerbe kerülő edzőgépeket. Ez magában foglalja a különböző kardió- és súlyzós eszközök szabad kiválasztását és kombinálását ahelyett, hogy kizárólag előre definiált csomagokra lennének korlátozva. Ez a fejlesztés nemcsak nagyobb szabadságot és személyre szabhatóságot biztosít a felhasználók számára, hanem lehetőséget teremt arra is, hogy mindenki a saját edzési céljaihoz, helyigényéhez és költségkeretéhez leginkább illeszkedő megoldást hozza létre.

Irodalom jegyzék:

Az alábbi forrásokat használtuk a projektünk elkészítéséhez:

- https://getbootstrap.com/
- https://material.angular.dev/
- https://laravel.com/
- https://en.wikipedia.org/wiki/Front-end_web_development